

भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

COURSE PROJECT REPORT | MECHANICS OF SOLIDS

Shape Analytics

An interactive environment for stress analysis:
half-space elasticity, analytical simulators
and a finite element solver

Submitted by

Abhinav Kumar Sudhir Choudhary	CO25BTECH11001
Rudra Thummar	CO25BTECH11024
Tilak Asodariya	CO25BTECH11028
Sriramnarayanan Mohan	CO25BTECH11027

Under the guidance of

Prof. Prakhar Gupta

Abstract

Shape Analytics is a browser-based tool for exploring the stress, strain and deformation of elastic solids. It combines three levels of model: (i) a three-dimensional stress field built by superposing the Boussinesq and Cerruti half-space solutions over a load painted on the surface of a cuboid, cylinder or sphere; (ii) nine analytical simulators for classical problems (Mohr's circle, beams, shaft torsion, axial loading, pressure vessels, thermal stress, failure theories, energy methods and column buckling); and (iii) a finite element solver for bars, beams, plane-stress problems and a three-dimensional block.

This report states the theory behind each tool, shows how it is turned into code, and tests the code against independent results. The simulators reproduce closed-form solutions to the printed precision in every default case (Table 5.2). The finite element solver reproduces exact bar and beam solutions, converges at the expected rate (the four-node element is within 0.3% of beam theory on eight elements through the depth), and recovers $K_t = 3.06$ for a hole in a wide plate against the classical value of 3.

Contents

Abstract i

1 Introduction and system overview 1

1.1 Motivation and objectives . . . 1

1.2 The software . . . 1

2 Stress theory and the 3D stress field 3

2.1 Stress at a point . . . 3

2.2 Half-space elasticity . . . 3

2.3 The 3D stress field pipeline . . . 4

2.4 Section analytics . . . 5

3 The analytical simulators 6

3.1 Framework . . . 6

3.2 Beams . . . 6

3.3 Shaft torsion and axial loading . . . 6

3.4 Pressure vessels and thermal stress . . . 7

3.5 Failure theories and energy methods . . . 7

3.6 Column buckling . . . 7

4 The finite element method 8

4.1 Method and implementation . . . 8

4.2 Elements and test problems . . . 8

4.3 The three-dimensional block . . . 8

5 Verification and results 10

5.1 Approach . . . 10

5.2 Default cases against closed forms . . . 10

5.3 Convergence of the finite element solver . . . 11

5.4 Worked example: a simply supported beam . . . 11

6 Discussion and conclusions 13

6.1 Limits of validity . . . 13

6.2 Conclusions . . . 13

Acknowledgements . . . 13

Chapter 1

Introduction and system overview

1.1 Motivation and objectives

Stress is not directly observable: it is a tensor with six independent components, and textbook solutions are closed forms for idealised geometries that hide the assumptions making them valid. The most instructive situations, such as a load on only part of a surface, have no simple closed form. This project builds an interactive tool that applies the governing equations to arbitrary inputs and states its assumptions. Its objectives are to

1. compute and display the stress, strain and deformation inside a solid loaded over a user-painted patch of its surface (the *3D stress field*);
2. provide analytical simulators for nine canonical problems, each showing its governing equations beside the result;
3. provide a finite element solver against which the analytical results can be compared;
4. verify every component against closed-form results, limiting cases and invariants.

All models are **linear elastic** with **small strain**, so superposition holds. The 3D field treats the body as an elastic half-space, which is exact for a body much larger than the loaded area; the finite element block (Section 4.3) measures the error otherwise. Plasticity, large rotation, contact and dynamics are out of scope.

1.2 The software

Shape Analytics is a browser application served by a small Python (FastAPI) server. It has three groups of tools, the **3D stress field** with section analytics, nine **analytical simulators**, and a **finite element tab**, all reached from one navigation bar (Figure 1.1).

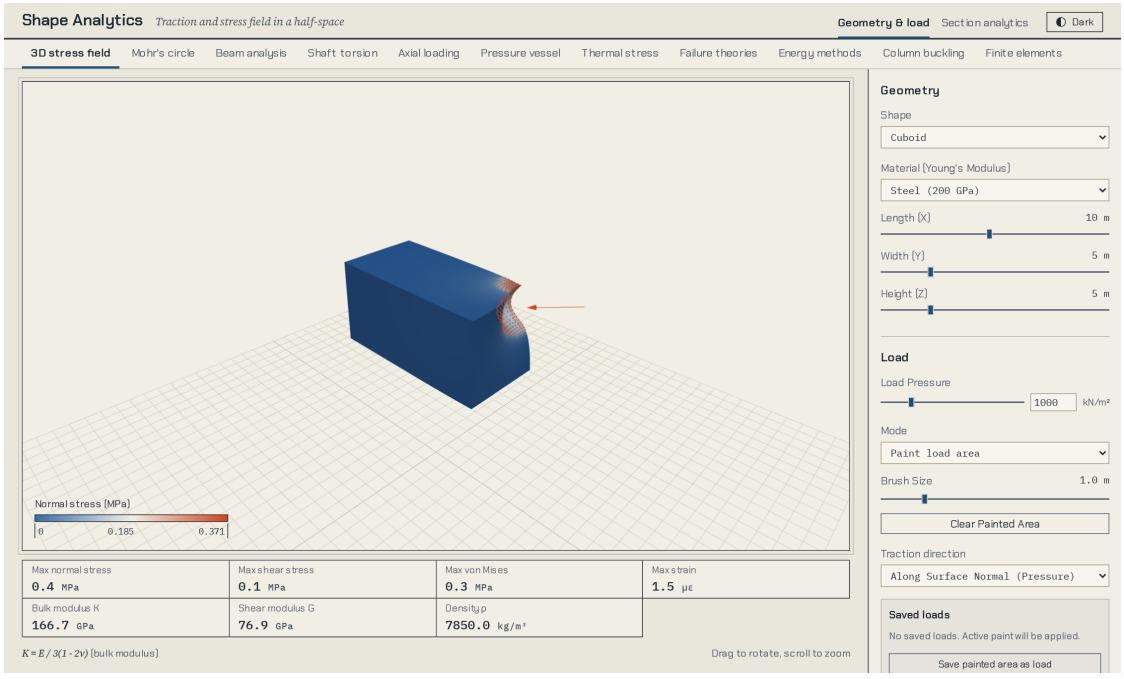


Figure 1.1. The 3D stress field. A load painted on the top-right edge of a steel cuboid produces the coloured normal-stress map; the readout cards give the maxima of the displayed measures.

File	Role
`index.html`, `style.css`	page structure, design tokens, light and dark themes
`app.js`	3D field: geometry, painting, loads, stress computation, section plots
`physics.js`	half-space stresses and displacements, principal stresses, von Mises, strain
`sims.js`	the nine simulators as configuration objects, and the framework that renders them
`fem.js`	the FEM tab: controls, mesh drawing, charts, explanation panels
`server.py`	`/api/traction`, `/api/principal`, `/api/section`, static files
`fem.py`, `fem3d.py`	finite element solvers (bar, beam, 2D, 3D brick) and `/api/fem/solve`

Conventions. Tension is positive; in half-space frames z points *into* the body. The 3D field works in SI units internally and shows MPa; the simulators use mm, N and MPa; the FEM tab uses mm, kN and MPa in the interface and SI in the solver.

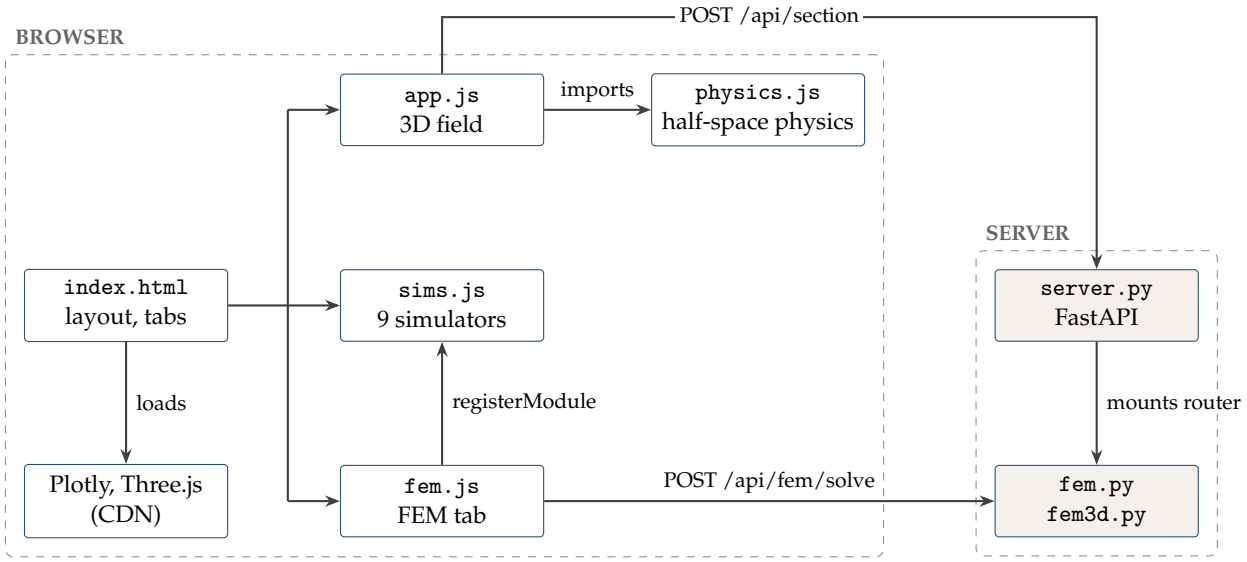


Figure 1.2. Architecture. The 3D field is computed in the browser (`physics.js`); only the section plots and the FEM solves call the server.

There is no build step: the scripts are ES modules loaded by `index.html`, and `sims.js` and `fem.js` register themselves into the same navigation bar (Figure 1.2). The 3D field runs entirely in the browser; the section plots and FEM solves go to the server because they evaluate thousands of points. The script `run_shape_analytics.sh` starts the server on port 8002, which also serves the page.

Chapter 2

Stress theory and the 3D stress field

2.1 Stress at a point

Theory. On a cut with unit normal $\mathbf{n}$ the traction is given by Cauchy's formula, and splits into normal and shear parts:

$$\mathbf{t}_n = \boldsymbol{\sigma}\,\mathbf{n}, \qquad \sigma_{nn} = \mathbf{n}\cdot\boldsymbol{\sigma}\mathbf{n}, \qquad \tau^2 = \|\mathbf{t}_n\|^2 - \sigma_{nn}^2. \tag{2.1}$$

Principal planes carry no shear, which gives the eigenproblem $\boldsymbol{\sigma}\mathbf{n} = \lambda\mathbf{n}$ with characteristic equation $\lambda^3 - I_1\lambda^2 + I_2\lambda - I_3 = 0$, where $I_1 = \operatorname{tr}\boldsymbol{\sigma}$, $I_2 = \tfrac{1}{2}[(\operatorname{tr}\boldsymbol{\sigma})^2 - \operatorname{tr}(\boldsymbol{\sigma}^2)]$ and $I_3 = \det\boldsymbol{\sigma}$. The roots $\sigma_1 \ge \sigma_2 \ge \sigma_3$ are real because $\boldsymbol{\sigma}$ is symmetric, and $\tau_{\max} = (\sigma_1 - \sigma_3)/2$. Two equivalent stresses compare a general state with a tension test:

$$\sigma_{\text{Tresca}} = \sigma_1 - \sigma_3, \qquad \sigma_{vM} = \sqrt{\tfrac{1}{2}\left[(\sigma_1-\sigma_2)^2 + (\sigma_2-\sigma_3)^2 + (\sigma_3-\sigma_1)^2\right]} = \sqrt{3J_2}. \tag{2.2}$$

The von Mises stress depends only on the deviator, so it vanishes in hydrostatic compression.

Implementation. The server (`/api/traction`, `/api/principal`) transcribes these formulas and uses `numpy.linalg.eigh`, which also returns principal directions. The browser needs principal stresses at tens of thousands of vertices per update, so `physics.js` uses the closed-form trigonometric root of the cubic, $\sigma_k = p + 2\sqrt{J_2/3}\cos(\phi - 2\pi k/3)$ with $\phi = \tfrac{1}{3}\arccos\left(3\sqrt{3}J_3/2J_2^{3/2}\right)$. The arccos argument is clamped to $[-1, 1]$, and the hydrostatic test is relative to the stress scale so that small states are not misclassified.

```
75 export function principalStresses(t) {
76     const [a, b, c, d, f, e] = t; // d = tauxy, f = tauxz, e = tauyz
77     const p = (a + b + c) / 3;
78     const sa = a - p, sb = b - p, sc = c - p;
79     const j2 = 0.5 * (sa * sa + sb * sb + sc * sc) + d * d + e * e + f * f;
80     // Hydrostatic state: the deviator is zero to rounding. The test is relative to the stress scale, so a
81     // genuinely tiny stress state (e.g. far from a load) is still resolved.
82     const scale2 = a * a + b * b + c * c + 2 * (d * d + e * e + f * f);
83     if (j2 <= 1e-24 * scale2) return [p, p, p];
84     const j3 = sa * (sb * sc - e * e) - d * (d * sc - e * f) + f * (d * e - sb * f);
85     const r = 2 * Math.sqrt(j2 / 3);
86     const phi = Math.acos(Math.max(-1, Math.min(1, (3 * Math.sqrt(3) * j3) / (2 * j2 ** 1.5)))) / 3;
87     return [0, 1, 2].map((k) => p + r * Math.cos(phi - (2 * Math.PI * k) / 3)).sort((m, n) => n - m);
88 }
89
90 export const vonMises = (t) => Math.sqrt(0.5 * ((t[0] - t[1]) ** 2 + (t[1] - t[2]) ** 2 + (t[2] - t[0]) ** 2 + 6 * (t[3] ** 2 + t[4]
   ↪  ** 2 + t[5] ** 2)));
```

`physics.js, lines 75–90`

Mohr's circle simulator. Rotating the element about z by θ moves the point $(\sigma_{x'}, \tau_{x'y'})$ round a circle of radius $\sqrt{(\frac{\sigma_x - \sigma_y}{2})^2 + \tau_{xy}^2}$ centred at $(\sigma_x + \sigma_y)/2$, at twice the rotation angle. `mohr.compute` in `sims.js` applies the transformation law, solves the 3D eigenproblem and draws all three circles; the safety factor is $\sigma_y/\sigma_{\text{eq}}$ (shown as ∞ for a hydrostatic state).

2.2 Half-space elasticity

Theory. A linear elastic body in equilibrium satisfies $\nabla\cdot\boldsymbol{\sigma} + \mathbf{b} = \mathbf{0}$, $\boldsymbol{\epsilon} = \tfrac{1}{2}(\nabla\mathbf{u} + \nabla\mathbf{u}^T)$ and $\boldsymbol{\sigma} = \lambda\operatorname{tr}(\boldsymbol{\epsilon})\mathbf{I} + 2G\boldsymbol{\epsilon}$. These have closed-form solutions only for special geometries; the simplest

useful one is the *half-space* $z > 0$ with a free surface. For a normal point force P (Boussinesq [3, 2]), at depth z and radius r with $R^2 = r^2 + z^2$,

$$\sigma_z = -\frac{3Pz^3}{2\pi R^5}, \qquad \tau_{rz} = -\frac{3Prz^2}{2\pi R^5}, \tag{2.3}$$

$$\sigma_r = \frac{P}{2\pi}\left[\frac{1-2\nu}{R(R+z)} - \frac{3r^2 z}{R^5}\right], \qquad \sigma_\theta = \frac{P(1-2\nu)}{2\pi}\left[\frac{z}{R^3} - \frac{1}{R(R+z)}\right]. \tag{2.4}$$

The solution satisfies $\nabla \cdot \boldsymbol{\sigma} = \mathbf{0}$, leaves the surface traction-free outside the load, and carries the full load through every horizontal plane, $\int_0^\infty \sigma_z \, 2\pi r \, dr = -P$.

Implementation. `boussinesqTensor` evaluates these in cylindrical form and rotates them to Cartesian axes. On the axis $\sigma_r = \sigma_\theta$, so the undefined azimuth does no harm.

```
export function boussinesqTensor(x, y, z, Pz, a, nu) {
    const ze = z + a;
    const r2 = x * x + y * y;
    const R = Math.sqrt(r2 + ze * ze);
    const R3 = R * R * R, R5 = R3 * R * R;
    const C = Pz / (2 * Math.PI);
    const sr = C * ((1 - 2 * nu) / (R * (R + ze)) - (3 * r2 * ze) / R5);
    const st = C * (1 - 2 * nu) * (ze / R3 - 1 / (R * (R + ze)));
    const sz = (-3 * Pz * ze * ze * ze) / (2 * Math.PI * R5);
    const trz = (-3 * Pz * ze * ze) / (2 * Math.PI * R5); // times r, split below
    if (r2 < 1e-16) return [(sr + st) / 2, (sr + st) / 2, sz, 0, 0, 0];
    const r = Math.sqrt(r2), c = x / r, s = y / r;
    return [sr * c * c + st * s * s, sr * s * s + st * c * c, sz, (sr - st) * c * s, trz * r * c, trz * r * s];
}
```

physics.js, lines 44–57

Tangential load (Cerruti [4]). Its closed form is simplest as a displacement field. `tangentialTensor` differentiates that displacement numerically (central step $h = 10^{-4}\sqrt{x^2 + y^2 + z_e^2}$) and applies Hooke's law, which guarantees a compatible strain. Since $\lambda \to \infty$ as $\nu \to 0.5$, ν is capped at 0.499 there.

Equivalent depth. A point load gives infinite stress beneath it, but a painted patch is a distributed pressure. The patch is represented by point loads at its mesh vertices, each shifted to a depth $z_e = z + a$, with a chosen from the patch geometry so that neighbouring loads overlap smoothly. This is numerical smoothing, not a physical parameter.

2.3 The 3D stress field pipeline

1. **Loads.** A pressure p on a patch of area A is a force $P = pA$. Each vertex is given a third of the area of every mesh triangle it belongs to, so the force does not depend on how finely the mesh is sampled.
2. **Local frames.** Each load uses a frame with $\mathbf{n}_z = -\mathbf{n}_s$ (into the body); the load direction splits into a normal part (Boussinesq) and a tangential part (Cerruti).
3. **Rotation and superposition.** Each point-source tensor is rotated to global axes, $\boldsymbol{\sigma}_g = \mathbf{R}\boldsymbol{\sigma}_l\mathbf{R}^T$, and the *tensors* from every point of every load are summed. Only then are principal stresses, von Mises stress and strain computed, because these are not linear in the tensor.
4. **Full-surface pressure.** When a normal pressure covers at least 99% of the surface area, the half-space model is replaced by the exact hydrostatic state $\boldsymbol{\sigma} = -p\mathbf{I}$, with volumetric strain $-p/K$.
5. **Display.** Four maps are offered: normal stress $\max(|\sigma_1|, |\sigma_3|)$, maximum shear $(\sigma_1 - \sigma_3)/2$, von Mises stress, and strain $\|\boldsymbol{\epsilon}\|$ from inverse Hooke's law. Displacements are the summed

Boussinesq–Cerruti displacements, exaggerated by a user factor. Strains above 1% are flagged as beyond small-strain theory.

```
505    const rot = (info, t) => { // local tensor -> global: R T R^T with R = [nx ny nz]
506        const a = [info.nx, info.ny, info.nz], out = [0, 0, 0, 0, 0, 0];
507        const L = [[t[0], t[3], t[4]], [t[3], t[1], t[5]], [t[4], t[5], t[2]]];
508        const pairs = [[0,0],[1,1],[2,2],[0,1],[0,2],[1,2]];
509        pairs.forEach(([p, q], k) => {
510            let s = 0;
511            for (let i = 0; i < 3; i++) for (let j = 0; j < 3; j++) {
512                s += L[i][j] * a[i].getComponent(p) * a[j].getComponent(q);
513            }
514            out[k] = s;
515        });
516        return out;
```

app.js, lines 505–516

2.4 Section analytics

The 3D map shows only the surface; the drawer below it shows the stress on a plane cut through the body (Figure 2.1). `compute_section` in `server.py` evaluates the same half-space tensors on a grid in the cut plane, vectorised with NumPy, takes all eigenvalues at once with `eigvalsh`, and blanks points outside the true shape (box, cylinder or sphere). The front end tags each request with a sequence number and ignores late replies, so dragging the slider never shows a stale picture.

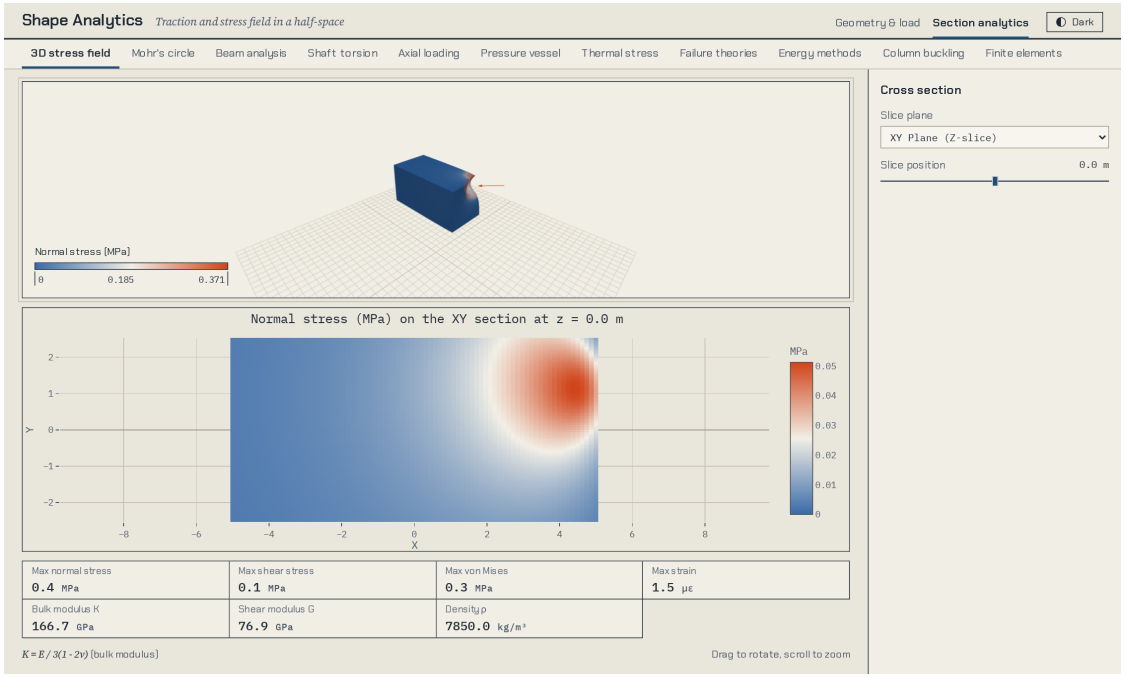


Figure 2.1. Section analytics: normal stress on a horizontal plane, evaluated on a grid by the server; points outside the body are blank.

Chapter 3

The analytical simulators

3.1 Framework

Each simulator in `sims.js` is a plain object: a list of `inputs` (sliders and selectors) and a function `compute(v)` returning a status banner, readouts, charts and the governing equations. The framework draws the controls, calls `compute` on every change, and adds an A/B comparison (a saved state drawn dotted), CSV/SVG/lab-report export, URL routing (`#beam`) and light/dark themes. `registerModule` lets another file add a tab (the FEM tab does this). Mohr's circle was described in Chapter 2; the other eight are summarised below.

3.2 Beams

Theory. Equilibrium gives $dV/dx = -w$ and $dM/dx = V$; Euler–Bernoulli theory gives $v'' = M/EI$, $\sigma = My/I$ and $\tau = VQ/(Ib)$. Timoshenko theory adds a shear deflection $V/(k_s GA)$, important only for short deep beams.

Implementation. Reactions follow from statics (simply supported or cantilever; point load, UDL and moment by superposition). V and M are evaluated at 401 points, M/EI is integrated twice with the trapezoidal rule, and the boundary conditions are then imposed. $Q(y)$ is integrated numerically, so rectangle, circle and tube need no separate formulas.

```
191        const slope0 = cumtrapz(x, Mx.map((q) => q / EI));
192        const raw = cumtrapz(x, slope0);
193        let vy;
194        if (v.type === 'ss') vy = raw.map((q, i) => q - (raw.at(-1) * x[i]) / L);
195        else vy = raw;
196        let shearDefl = x.map(() => 0);
197        if (v.theory === 'tbt') {
198            const ys = cumtrapz(x, Vx.map((q) => q / (sec.ks * G * sec.A)));
199            shearDefl = v.type === 'ss' ? ys.map((q, i) => q - (ys.at(-1) * x[i]) / L) : ys;
200        }
```

`sims.js, lines 191-200`

3.3 Shaft torsion and axial loading

Theory. **Torsion:** plane sections rotate rigidly, so $\tau(r) = Tr/J$ and $\phi = TL/GJ$ with $J = \pi(D^4 - d^4)/32$ and $G = E/2(1+\nu)$. The simulator also draws a 3D shaft whose end face is coloured by $\tau \propto r$. **Axial:** for a segmented bar the internal forces come from cuts and each segment stretches by $N_i L_i/(EA_i) + \alpha \Delta T L_i$. With both ends fixed the wall reaction X is the redundant, found from zero total elongation:

$$X = -\frac{\sum_i N_i^{\text{free}} L_i/(EA_i) + \alpha \Delta T \sum_i L_i}{\sum_i L_i/(EA_i)}. \tag{3.1}$$

A heated fixed bar with no load gives $\sigma = -E\alpha\Delta T$ (-120 MPa for steel at 50°C).

3.4 Pressure vessels and thermal stress

Theory. **Vessels:** thin wall $\sigma_\theta = pr/t$ (cylinder) or $pr/2t$ (sphere). Thick wall uses the Lamé solution, for a cylinder

$$\sigma_{r,\theta} = A \mp \frac{B}{r^2}, \qquad A = \frac{p_i a^2 - p_o b^2}{b^2 - a^2}, \quad B = \frac{(p_i - p_o)a^2 b^2}{b^2 - a^2}, \tag{3.2}$$

and the analogous $1/r^3$ form for a sphere. The thin-wall error against Lamé is reported. **Thermal:** a temperature change ΔT fully restrained in one, two or three directions gives $\sigma = -E\alpha\Delta T$, $-E\alpha\Delta T/(1-\nu)$ or $-E\alpha\Delta T/(1-2\nu)$; a partial restraint scales the stress.

3.5 Failure theories and energy methods

Theory. **Failure:** Rankine ($\max|\sigma_i|$), St Venant ($\epsilon_1 E$), Tresca ($\sigma_1 - \sigma_3$) and von Mises are each compared with the strength σ_{cr}; the theory with the lowest safety factor governs. Their envelopes in the (σ_1, σ_2) plane (square, skewed square, hexagon, ellipse) are drawn with the current state. **Energy:** the strain energy U of a bar, shaft or beam (bending and shear) is evaluated in closed form; Castigliano's theorem $\delta = \partial U/\partial P$ is checked by *numerical* differentiation against the closed-form displacement, and Maxwell–Betti reciprocity is checked on a simply supported beam.

3.6 Column buckling

Theory. Euler's load is $P_{cr} = \pi^2 EI/(KL)^2$ with $K = 1$, 0.699, 0.5, 2 for the four end conditions and I the weak-axis value. It holds only for slenderness $\lambda = KL/r > \lambda_c = \pi\sqrt{E/\sigma_y}$; shorter columns use the Johnson parabola $\sigma_{cr} = \sigma_y[1 - \sigma_y\lambda^2/(4\pi^2 E)]$, which meets Euler's curve at λ_c.

Chapter 4

The finite element method

4.1 Method and implementation

The half-space solution is exact only for an infinite body and the simulators only for simple members. The finite element method solves the same equations on a body of any shape:

1. mesh the body into elements joined at nodes, whose displacements are the unknowns;
2. interpolate inside each element, $\mathbf{u} = \sum_i N_i \mathbf{u}_i$, so that $\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u}_e$ and $\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\epsilon}$;
3. form $\mathbf{k}_e = \int \mathbf{B}^T\mathbf{D}\mathbf{B}\, dV$ by Gauss quadrature and assemble the global $\mathbf{K}$;
4. remove supported degrees of freedom, solve $\mathbf{K}\mathbf{u} = \mathbf{f}$, and recover stresses and the reactions $\mathbf{R} = \mathbf{K}\mathbf{u} - \mathbf{f}$.

```
40 def solve_system(K, f, fixed):
41     """Partition out the constrained DOFs (supports) and solve K_ff u_f = f_f; also return reactions."""
42     n = K.shape[0]
43     free = np.setdiff1d(np.arange(n), fixed)
44     u = np.zeros(n)
45     u[free] = np.linalg.solve(K[np.ix_(free, free)], f[free])
46     return u, K @ u - f
```

`fem.py, lines 40-46`

4.2 Elements and test problems

- **Tapered bar** (2-node elements): exact solution $u = \frac{P}{Em} \ln \frac{A(x)}{A_0}$; the FEM converges as h^2.
- **Beam** (Hermite cubic, deflection and slope per node): nodal deflections are exact for any mesh, because the cubic solves $EIv'''' = 0$ between loads.
- **Plane stress** with $\mathbf{D} = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}$: the constant-strain triangle (CST) and the four-node quadrilateral (Q4, 2×2 Gauss points, stresses extrapolated to nodes and averaged). Two problems are solved: a cantilever under end shear (compared with Euler–Bernoulli and Timoshenko), and a quarter plate with a hole in tension, compared with Kirsch's $K_t = 3$ [8].

4.3 The three-dimensional block

Theory. A cuboid with a square patch pressure q on its top is meshed (one quarter, using symmetry) with 8-node bricks. The sides and unloaded top are free: the weak form makes their traction zero automatically, which is exactly what the half-space solution cannot do. The reference is Newmark's solution for the vertical stress under a rectangular patch [7].

Implementation. All bricks are identical, so one 24×24 stiffness matrix is reused. The system is solved *matrix-free* by Jacobi-preconditioned conjugate gradients: $\mathbf{K}\mathbf{x}$ is assembled brick by brick by gather, multiply and scatter (`np.bincount`), so $\mathbf{K}$ is never formed.

```
def cg(matvec, b, dinv, free, tol=1e-9, maxit=6000):
    x = np.zeros_like(b); r = np.where(free, b, 0.0); z = dinv * r; p = z.copy(); rz = r @ z; b0 = np.linalg.norm(r) or 1.0
    for it in range(1, maxit + 1):
        Ap = np.where(free, matvec(p), 0.0)
        alpha = rz / (p @ Ap)
        x += alpha * p; r -= alpha * Ap
        if np.linalg.norm(r) < tol * b0:
            return x, it, float(np.linalg.norm(r) / b0)
        z = dinv * r; rn = r @ z; p = z + rn / rz * p; rz = rn
    return x, maxit, float(np.linalg.norm(r) / b0)


# ---------------------------------------------------------------- half-space reference (tension positive)
```

`fem3d.py, lines 54–66`

User interface. `fem.js` sends the inputs of the five problems to `POST /api/fem/solve` and draws the returned mesh coloured by stress, the charts, a FEM-versus-reference table and a panel describing what the solver did with the current numbers (Figure 4.1).

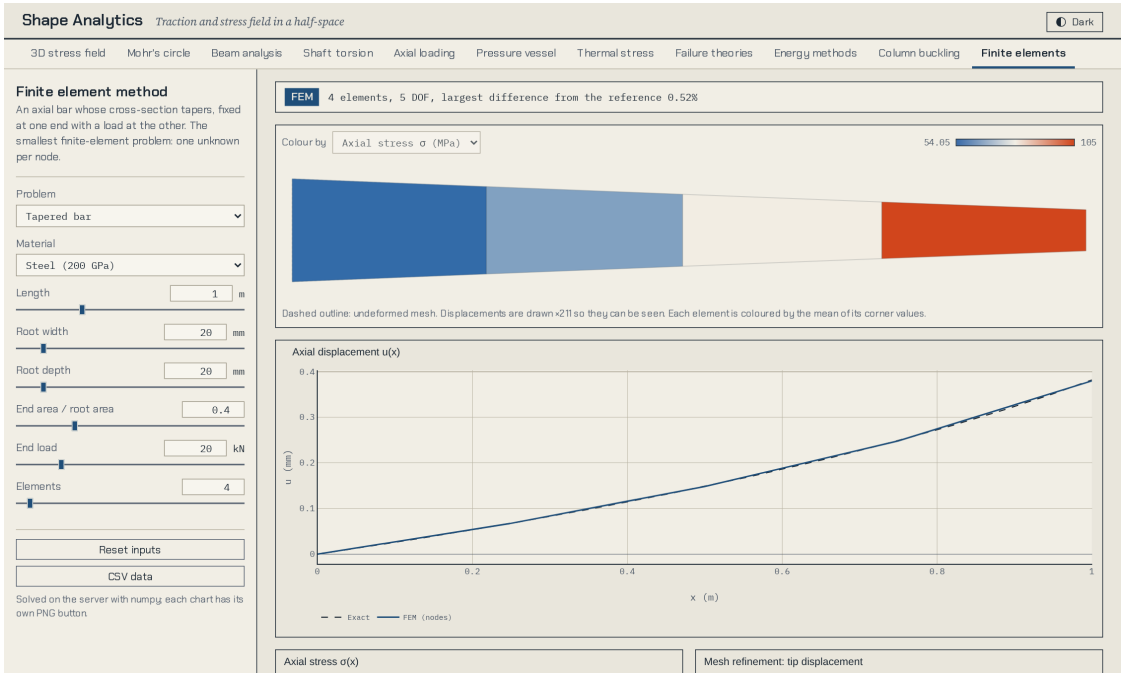


Figure 4.1. The finite element tab for the tapered bar: mesh coloured by axial stress and displacement against the exact solution. With four elements the largest error is 0.52%.

Chapter 5

Verification and results

5.1 Approach

A simulator can be wrong in ways that look reasonable. Each component was therefore compared with something independent: a closed form, a limiting case with a known answer, an invariant that must hold for any input, or a second implementation of the same physics. Table 5.1 summarises the checks recorded during development (the test scripts were later removed from the repository at the owner's request); Sections 5.2 and 5.3 were re-run for this report.

Table 5.1. Checks recorded during development.

Component	Checks	What was verified
`physics.js`	54	principal-stress invariants for 4000 random tensors over 12 decades; Boussinesq and Cerruti satisfy $\nabla\cdot\boldsymbol{\sigma}=\mathbf{0}$; $\int\sigma_z\,dA=-P$; free surface; far-field decay
`server.py`	40	eigenvalues against NumPy; $\lvert\mathbf{t}\rvert^2=\sigma_{nn}^2+\tau^2$; section linear in load and symmetric; browser and server agree to 2×10^{-4}
Simulators	110	each closed form plus limiting cases (uniaxial, hydrostatic, pure shear, zero load, thin-wall limit, slender and stocky columns)
Simulators, random	22 500	2500 random or extreme inputs per simulator: no exception, no NaN
FEM	43	patch test to 10^{-8}, exact beam nodes, reactions equal loads, block under full pressure, 116 extreme inputs

5.2 Default cases against closed forms

Each simulator was opened with its default inputs and the displayed result compared with an independent NumPy evaluation of the closed form ($E=200$ GPa, $\nu=0.3$). Every quantity agrees to the printed precision (Table 5.2).

Table 5.2. Application output for the default inputs against an independent closed-form evaluation.

Simulator	Quantity	Application	Closed form	Unit
Mohr, $(80,-20)$, $\tau_{xy}=-39$	σ_1 / $\tau_{\max}$	93.41 / 63.41	93.41 / 63.41	MPa
	von Mises σ_{eq}	113.9	113.9	MPa
Beam, 80×160 mm, $P=10$ kN, $L=3$ m	$M_{\max}$	7.50	7.50	kN m
	$\sigma_{\max}$ / $\delta_{\max}$	21.97 / 1.03	21.97 / 1.03	MPa / mm
Shaft, $D=50$ mm, $T=800$ N m	$\tau_{\max}$ / twist	32.59 / 1.17	32.59 / 1.17	MPa / deg
Axial, three segments	$\sigma_{\max}$ / elongation	87.50 / 0.338	87.50 / 0.338	MPa / mm
Vessel, $a=200$, $t=12$, $p=10$ MPa	Lamé hoop at bore	171.8	171.8	MPa
Thermal, $\Delta T=60$ K, restrained	$-E\alpha\Delta T$	−144.0	−144.0	MPa
Failure, $(150,60,0)$	von Mises	130.8	130.8	MPa
Energy, cantilever, $P=2$ kN	U	26.53	26.53	J
Buckling, 30×50 mm, $L=2$ m	P_{cr}	55.52	55.52	kN

5.3 Convergence of the finite element solver

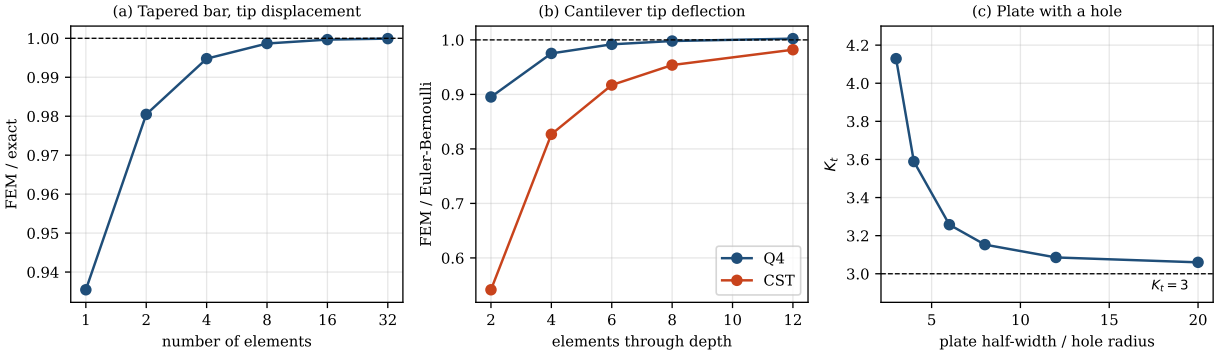


Figure 5.1. FEM convergence. (a) Tapered bar tip displacement over exact. (b) Plane-stress cantilever tip deflection over Euler–Bernoulli, for Q4 and CST. (c) Stress concentration of a plate with a hole against plate half-width over hole radius; dashed line: Kirsch $K_t = 3$.

- **Bar** (a): ratio 0.935 with one element to 0.9999 with thirty-two; the error falls about fourfold per mesh doubling, as expected for linear elements.
- **Cantilever** (b): Q4 reaches 0.998 with eight elements through the depth, CST only 0.954; both are too stiff in bending on coarse meshes.
- **Hole** (c): K_t falls from 4.13 for a plate three radii wide to 3.06 for twenty, approaching Kirsch's value from above as the finite-width effect disappears.
- **Block**: the centre stress is 1.26, 1.06 and 1.03 times the half-space value for 2×3, 4×4 and 8×8 m blocks, so the 3D field's half-space model is good to about 3% once the body is several times larger than the patch.

5.4 Worked example: a simply supported beam

A steel beam ($E = 200$ GPa), section 80×160 mm, span 3 m, central load 10 kN. By hand, $I = bh^3/12 = 27.3 \times 10^6$ mm^4, so

$$M_{\max} = \frac{PL}{4} = 7.50 \text{ kN m}, \qquad \sigma_{\max} = \frac{M_{\max} c}{I} = 21.97 \text{ MPa}, \qquad \delta_{\max} = \frac{PL^3}{48EI} = 1.03 \text{ mm},$$

and the peak shear stress $1.5V/A = 0.59$ MPa is 37 times smaller than the bending stress. The application gives exactly these values and a safety factor of $250/21.97 = 11.4$ (Figure 5.2).

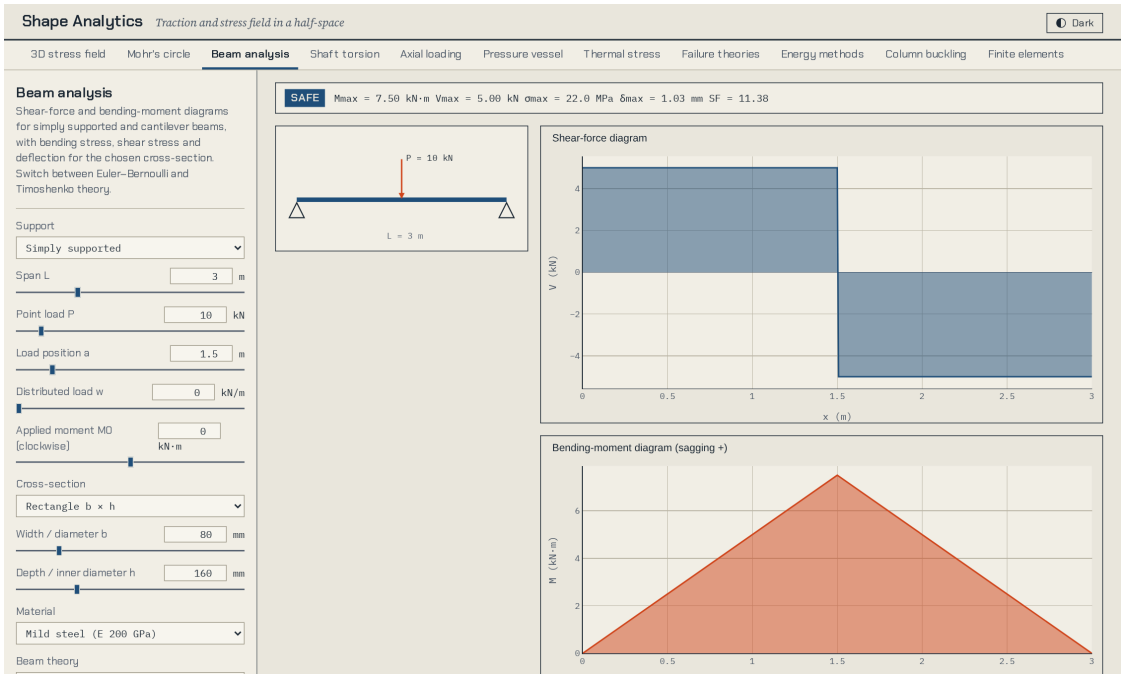


Figure 5.2. The beam simulator for the worked example: piecewise-constant shear force and triangular bending moment peaking at mid-span.

Chapter 6

Discussion and conclusions

6.1 Limits of validity

- **Linear elastic, small strain** everywhere: no plasticity, large rotation, contact or dynamics. Rubber in the 3D tab can exceed 100% strain, where results are indicative only.
- **Half-space** (3D field, sections): an infinite body, so a finite body's sides and base are not traction-free; the FEM block quantifies the error (Section 5.3).
- **Finite elements**: plane stress only; CST and Q4 are stiff in bending until refined; bricks lock for nearly incompressible materials; the dense 2D solver is limited to about 3600 degrees of freedom.
- **Environment**: Three.js and Plotly load from CDNs, so the interface needs network access.

6.2 Conclusions

1. The project combines half-space elasticity, closed-form simulators and finite elements in one interface, so a result obtained one way can be checked another.
2. All nine simulators reproduce their closed-form solutions to the printed precision (Table 5.2) and pass the recorded limiting-case and invariance tests.
3. The FEM solver converges monotonically: to 0.01% for the bar with 32 elements, within 0.3% of beam theory for the Q4 cantilever, and $K_t = 3.06$ for a hole in a wide plate against the classical 3.
4. The half-space approximation of the 3D field is good to about 3% for bodies several times larger than the loaded area.

Acknowledgements

We are sincerely grateful to **Prof. Prakhar Gupta** for giving us the opportunity to work on this project. His ideas shaped the project from the start: the discussions with him while we were deciding what to build gave us its direction and much of its scope. We thank him for his guidance, his encouragement and his trust in letting us develop the idea in our own way.

Abhinav Kumar Sudhir Choudhary, Rudra Thummar,
Tilak Asodariya and Sriramnarayanan Mohan

Bibliography

[1] S. P. Timoshenko and J. N. Goodier, *Theory of Elasticity*, 3rd ed., McGraw-Hill, 1970.

[2] K. L. Johnson, *Contact Mechanics*, Cambridge University Press, 1985.

[3] J. Boussinesq, *Application des potentiels à l'étude de l'équilibre et du mouvement des solides élastiques*, Gauthier-Villars, 1885.

[4] V. Cerruti, "Ricerche intorno all'equilibrio dei corpi elastici isotropi," *Atti Accad. Lincei*, 1882.

[5] R. C. Hibbeler, *Mechanics of Materials*, 10th ed., Pearson, 2017.

[6] R. D. Cook, D. S. Malkus, M. E. Plesha and R. J. Witt, *Concepts and Applications of Finite Element Analysis*, 4th ed., Wiley, 2002.

[7] N. M. Newmark, "Simplified computation of vertical pressures in elastic foundations," *Univ. Illinois Eng. Exp. Station Circular* 24, 1935.

[8] G. Kirsch, "Die Theorie der Elastizität und die Bedürfnisse der Festigkeitslehre," *Z. VDI* 42, 1898.